Laboratório de Concorrência · Aula 10

Pedro Massaki Carnelossi · RA 24036481-2 · Turma ESOFT6S-B
Gerado em 23/10/2026, 16:24:48 · tempo registrado no cronômetro: 19:37

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Cooperação é quando uma tarefa precisa esperar outra terminar algo antes de continuar. Exemplo: o garçom só leva o prato depois que o cozinheiro termina de prepará-lo. Competição é quando duas tarefas querem o mesmo recurso

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

O esperado seria 2.000.000. Mas, como as threads não estão sincronizadas, acho que o valor vai sair menor e diferente a cada execução.

1ª execução (n = 1.000.000)

não respondido

2ª execução (n = 1.000.000)

não respondido

3ª execução (n = 1.000.000)

não respondido

Saída do programa

não respondido

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram, e cada execução deu um valor diferente. Quem decide a ordem em que as threads executam é o escalonador, e essa ordem muda a cada execução. Por isso a quantidade de atualizações perdidas também muda.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

O contador vale 10.

A lê 10.
B lê 10.
A soma e fica com 11 no registrador.
B soma e também fica com 11.
A grava 11.
B grava 11 por cima.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, deu 2000 (correto). A primeira thread termina antes de a segunda começar, então as duas quase nunca se sobrepõem. Isso é perigoso porque o erro some nos testes pequenos e só aparece em produção, sob carga. Fica intermitente e difícil de reproduzir.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

não respondido

Saída do programa

não respondido

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Só 1 ordem (AAABBB) dá o resultado “correto”, 8.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Ordem ABBBAA:

A lê 3.
B lê 3, calcula 6 e grava 6.
A calcula 3+1 = 4 e grava 4.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A versão sem sincronização deve ser a mais rápida, mas incorreta. Entre as corretas, acho que AtomicInteger será mais rápida que synchronized.

Saída do programa

não respondido

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

synchronized e AtomicInteger deram exatamente 2.000.000. A versão sem sincronização deu menos. A versão correta é mais lenta porque cada incremento paga um custo de coordenação: adquirir e liberar a trava, ou a instrução atômica com sincronização de cache entre os núcleos. Além disso, as threads esperam uma pela outra.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não resolve. Cada saldo fica atômico sozinho, mas a transferência são duas operações: debitar uma conta e creditar a outra. Entre as duas, outra thread pode ver um estado inconsistente, com o dinheiro “sumido”, ou intercalar outra operação no meio. É preciso tornar a operação inteira atômica: travar as duas contas, sempre na mesma ordem para evitar impasse, ou usar uma transação.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não vão aparecer. A fica com X esperando Y, e B fica com Y esperando X.

Saídas (antes e depois da correção)

não respondido

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A pegou X e esperava Y. B pegou Y e esperava X. Cada uma segura o que a outra precisa e nenhuma solta o que tem, então formou-se uma espera circular.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Com a mesma ordem, quem pega X primeiro pega Y em seguida. A outra thread espera já no X, sem segurar nada, então não existe ciclo de espera. Exemplo real: duas transferências bancárias simultâneas, A→B e B→A, em que cada uma trava primeiro a própria conta de origem.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Tempos obtidos: 1 trabalhador ≈ 600 ms, 3 ≈ 200 ms, 6 ≈ 100 ms.

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

não respondido

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

O tempo é aproximadamente [6 ÷ trabalhadores] × 100 ms, porque as tarefas rodam em paralelo em “rodadas”. O programa não precisa de trava porque nenhuma variável é compartilhada. Cada número de tarefa é entregue pelo canal a um único trabalhador, e o próprio canal faz a sincronização.

Saídas (antes e depois da troca)

não respondido

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

Num canal sem buffer, quem envia fica bloqueado até alguém receber. A única goroutine (main) fica parada no envio e nunca chega à linha que receberia, então é um impasse. Com espaço para 1 mensagem, o envio deposita o valor e segue sem esperar. Depois a mesma goroutine recebe.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Acho que não compila. O Rust não deixa a thread alterar uma variável de main sem proteção.

Saídas (o erro e o resultado com move)

não respondido

Saída do programa

não respondido

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

i32 implementa Copy, então move dá à thread uma cópia do número. A thread soma 1 na cópia dela, e a variável contador de main continua 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java, o erro só apareceu em execução, com resultado errado e intermitente. Em Rust, ele foi barrado na compilação. A vantagem é que uma corrida de dados nunca chega à produção. O custo é que o código fica mais verboso (Arc, Mutex, lock, move), tem uma curva de aprendizado maior e é preciso “brigar” com o compilador.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Saída: A, D, C, B e depois E: timer de 10 ms rodou depois de ~300 ms.

Saída do programa

não respondido

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A e D são código síncrono e saem na hora. C é uma promessa e vai para a fila de microtarefas, que roda logo após o código atual. B é um setTimeout, que vai para a fila de tarefas (macrotarefas) e só roda depois das microtarefas, mesmo com tempo 0.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

Só existe uma thread, e ela ficou presa 300 ms no while. O timer de 10 ms estava pronto, mas só pôde rodar quando a thread ficou livre. Num servidor Node.js, um cálculo pesado trava todas as requisições de todos os usuários durante esse tempo. A solução é jogar esse cálculo para worker threads ou para outro serviço.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Condição de corrida. As duas compras leem “1 lugar disponível” antes de qualquer uma gravar, e o mesmo lugar é vendido duas vezes. Para evitar, a verificação e a reserva precisam ser uma operação atômica. Pode ser uma transação no banco com UPDATE ... SET vagas = vagas - 1 WHERE vagas > 0, conferindo quantas linhas foram afetadas, ou uma trava/fila por evento.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que só 1 das 20 ordens possíveis da Figura 13.1 dá o resultado “correto”, e que o erro some com n = 1000 e só aparece com volume.